KVDK Cleaner 设计文档

KVDK 后台 Cleaner 模块主要负责清理过时数据（outdated record）[1]。Cleaner 需要解决的三大问题为：

1. 如何快速发现过时数据
2. 如何快速处理过时数据
3. 避免前台发生较大的延迟抖动

针对上述的三大问题，目前的方案是 Cleaner 通过扫描 Hash Table 和 Collection Pool[2]找到并清理过时数据。当 engine 中存在大量的过期数据时，为了快速清理掉它们，Cleaner 将开启多线程模式批量扫描和清理过期数据以快速释放 PMEM 空间；当 engine 中存在少量的过期数据时，Cleaner 只有一个线程每隔 1s 扫描 Hash Table 和 Collection Pool。第二章节将会阐述设计思路和具体的实现代码。

该设计文档分文四个章节，第一章节展示了 KVDK Cleaner 模块的整体架构图，以及和前台用户的交互过程。第二章详细说明了 Cleaner 的设计思路和具体实现。第三章节展示了 Cleaner 的性能。第四章节探讨了 Cleaner 未来的一些可优化点。

1 Cleaner 整体架构

Figure1 展示了后台 Cleaner 模块的整体结构，以及前台用户操作和后台 Cleaner 之间的交互。前台用户执行写操作时，除了更新 Hash Table 和写入新数据之外，还会辅助后台 Cleaner 帮忙清理一些过时数据，具体做法为：用户更新某个数据的值（基于 MVCC[3]实现），需要扫描该 record 的版本链，找到不在用户设置的 snapshot[4] 上的旧数据，把它从版本链上摘除后放入到该线程的缓存队列中， 同时从缓存队列中试图清理一个过时数据

[1] 已经过期数的据，被删除的数据，以及被更新的老数据统称为过时数据

[2] 用户创建的 collection 的集合

[3] MVCC: Multi-Version Concurrency Control, 多版本控制，用于快照读和解决读写冲突等。

[4] Snapshot(快照)：某一时刻 engine 中的所有数据的映像。

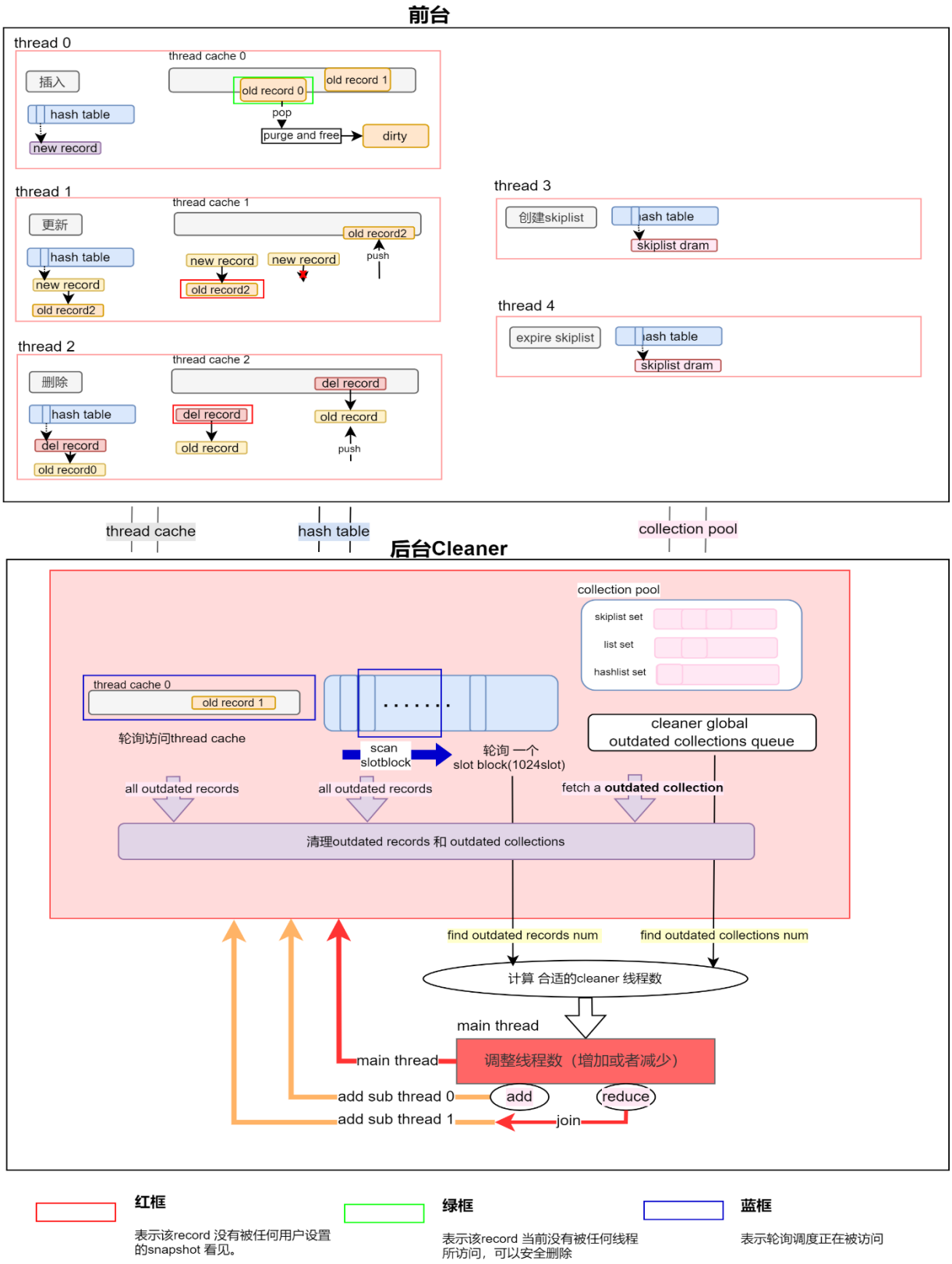

Figure 1： Cleaner 整体架构图

（图中绿框所示）。KVDK engine 中的 Collection Pool 记录了用户创建过的且还未过时的所有 collections（skiplist, list, hashlist），Collection Pool 有两点作用：

1. 断电恢复数据，记录已恢复的 collections。
2. 后台 Cleaner 快速找到过时数据，详细原因请见第 2.2 章节。

后台 Cleaner 主要工作是找到过时数据，并清理掉它们。如 Figure1 所示，后台通过三种途径寻找过期数据：

1. 从前台线程的缓存队列中获取过时数据，这些过期数据已经无法从 Hash Table 上查询到，Cleaner 只负责清理这些数据空间
2. 扫描 Hash Table 找到一些过时数据，比如 collection 中某个被删除的元素，或者是非 collection 类型的过期元素。Cleaner 首先要将这些数据从 Hash Table，以及从 collection 链表上摘除，其次再去释放数据空间。
3. 扫描 Collection Pool 找到过时 collection，比如用户删除的 collection，或者是生命周期结束的 collection。与处理 Hash Table 上的过时数据类似，Cleaner 同样要将过时的 collection 从 Hash Table 上摘除，不同的是，过时 collection 除了要从 Collection Pool 中挪走， 还需要特殊处理其版本链（参见第 2.2 章节详细说明）

2 Cleaner 详细设计和实现

2.1 Cleaner 获取前台线程缓存队列数据

让我们先回顾 Figure1 架构图中前台线程缓存队列辅助后台 Cleaner 的逻辑：每个写线程都要遍历一遍当前线程访问的数据的历史版本链，找到历史版本链中的不在任何用户设置的 snapshot 中的旧数据，从版本链上摘除并放入到该线程的缓存队列中，然后从缓存队列中试图释一个 outdated data。你可能会产生疑惑：为什么从版本链上摘除后，不能直接释放其空间，而是要放到前台线程缓存队列中，且缓存队列试图释放队队头 outdated data 的空间？这里涉及到 KVDK 内部避免读写冲突的机制：KVDK 内部也有一个 snapshot，为了区分用户设置的 snapshot，我们将内部 snapshot 定义为 token。当某时刻有多个线程同时读写某一数据时，为了避免读写冲突，KVDK 内部采用每个线程持有一个本地 token 的机制来保证事务隔离性。更进一步来说，当同时执行读写操作的线程释放了其持有的 token 后，才可以安全释放过时数据的空间。

如果某段时期内，前台没有发生任何读写操作，那么前台缓存队列中的过时数据的清理依赖于后台 Cleaner。后台 Cleaner 线程每隔一个 slot block 周期轮询收集一个前台线程缓存队列中数据。

2.2 Cleaner 扫描 Hash Table

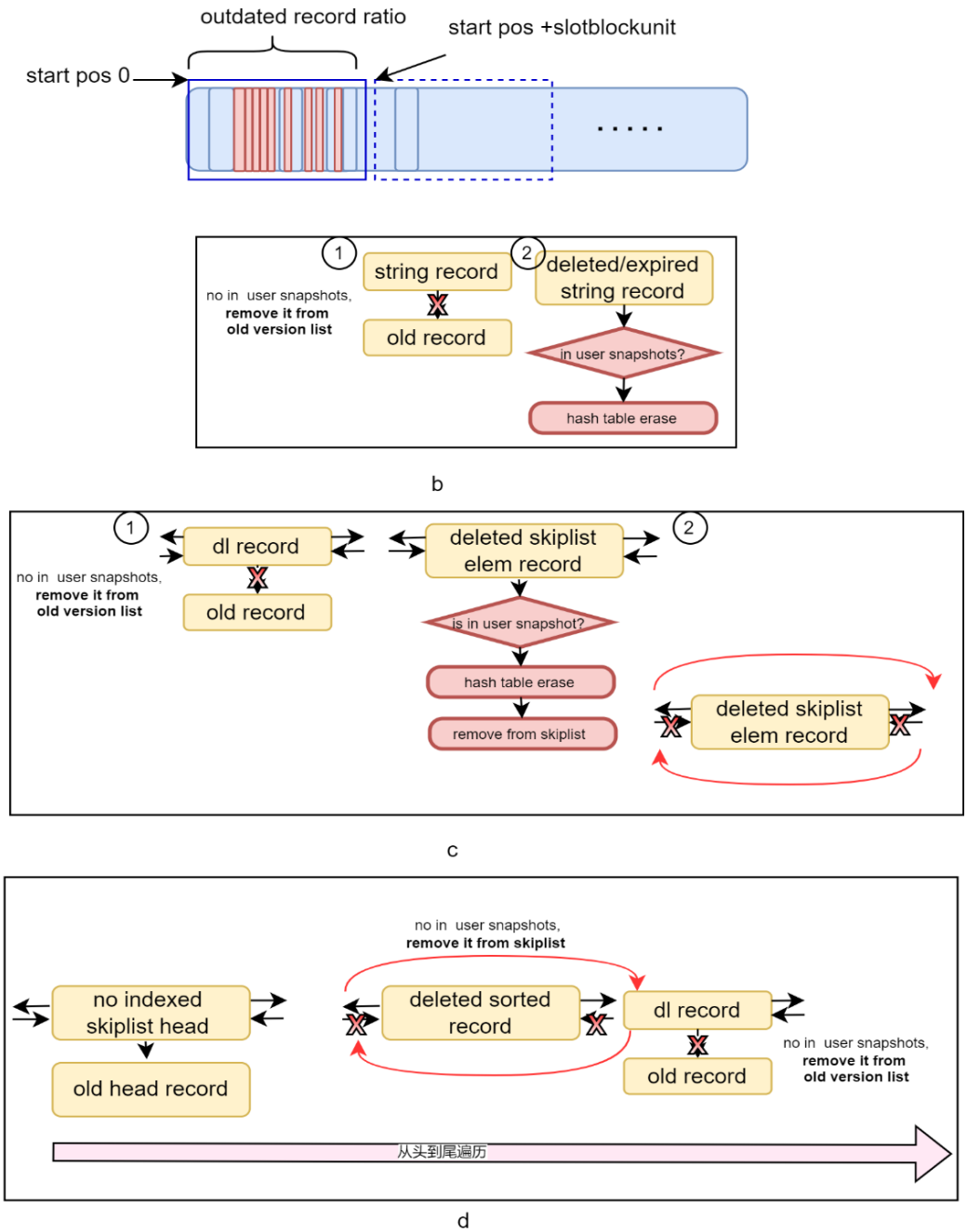

Figure 2: Scan Hash Table

Cleaner 采用轮询机制以 SlotBlockUnit (1024 个 slot)为步长扫描 Hash Table。扫描 Hash Table 可以很方便的找到过时 string 类型数据和 Collection 中过时元素(该 Collection 元素的索引在 Hash Table 上)。

对于 String 类型数据，因为数据之间的结构是独立的，且每个 String 数据都在 Hash Table 上，所以如果发现 Hash Table 上的 string 是已删除或者已过期，且不在任何用户设置的 snapshot 中，则直接从 Hash Table 上摘除；如果 Hash Table 上的 string 数据不是过时数据，那么则遍历其版本链，找到不在 Snapshot 上的旧数据，断开其版本链，如图 2b 所示。

对于 skiplist 类型的过时元素数据，与 string 类型不同的地方在于，由于 skiplist 是基于双链表实现，元素还需要从双链表上摘除，如图 2c 所示。

需要格外说明：KVDK 支持两种模式的 skiplist，一种是 skiplist 链上的所有元素都有 Hash Table 索引(WithHashIndex Skiplist)，另一种是 skiplist 链上只有头节点有 Hash Table 索引，简称（NoHashIndex Skiplist）。因此扫描 Hash Table 的过程中如果遇到 NoHashIndex skiplist，需要遍历该 skiplist 找到被删除的数据并清理它们的空间，如图 2d 所示。

2.3 Cleaner 扫描 Collection Pool

扫描 Hash Table 可以帮助我们快速的找到 string 数据和 collection 中被删除的元素，但是碰到 Figure 3 的场景时可能引发无法及时找到过时 collections 从而导致无法及时清理几十 G 的空间：当用户场景中存在少量但元素数量较多的 NoHashIndex Skiplist 时，由于 Hash Table 中过时的 string 数据，因此 Cleaner 只会有一个主线程在干活。如果该主线程只通过扫描 Hash Table 找到过时的 NoHashIndex Skiplist，可能需要经历几十秒或者几百秒的时间（包含对非过时的 NoHashIndex Skiplist 的遍历，Figure 2d 所示）。显然这是无法接受的情景，因此为了能够及时发现过时 collections，我们采用扫描 Collection Pool 方式实现快速找到过时 collections，具体的设计如下：

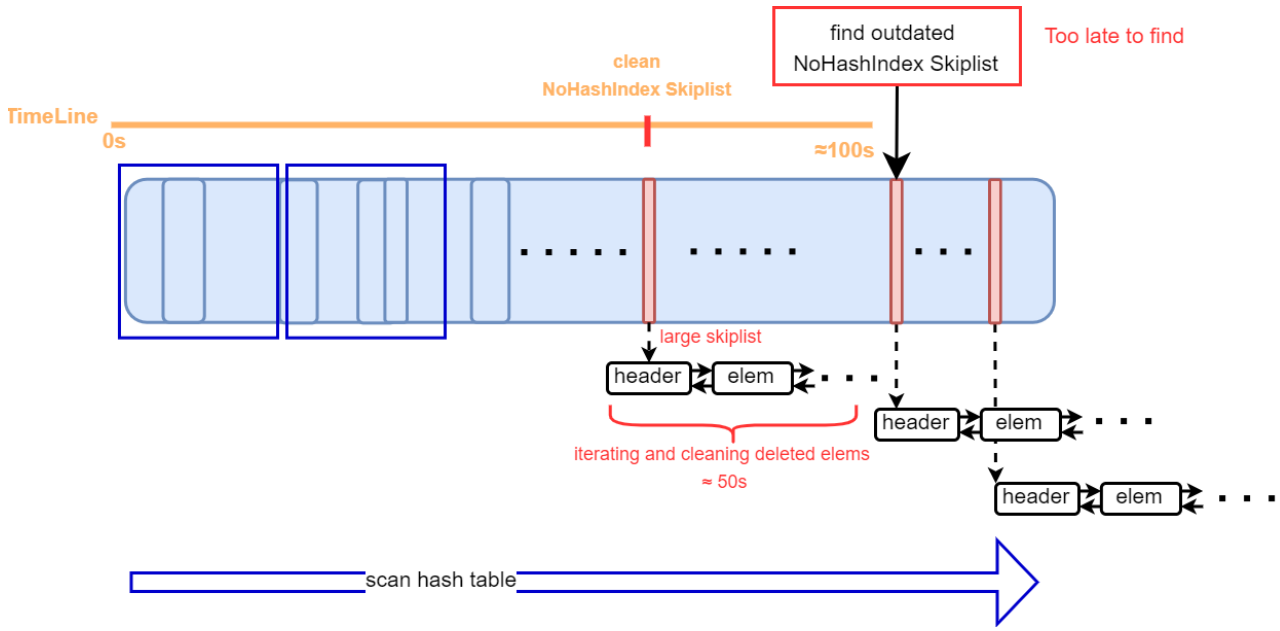

Figure 3

1. Collection Pool 是三种 Collection 类（Skiplist, List, HashList）的集合。Collection 集合是按照过期时间排序。前台用户创建 collection 后，将会被添加到 Collection Pool 中；用户设置 collection 过期时间或者删除 collection 的时候，该 Collection Pool 根据新的过期时间更新排序。按照过期时间排序的好处是 Cleaner 扫描 Collection Pool 时能快速找到过时 collection。

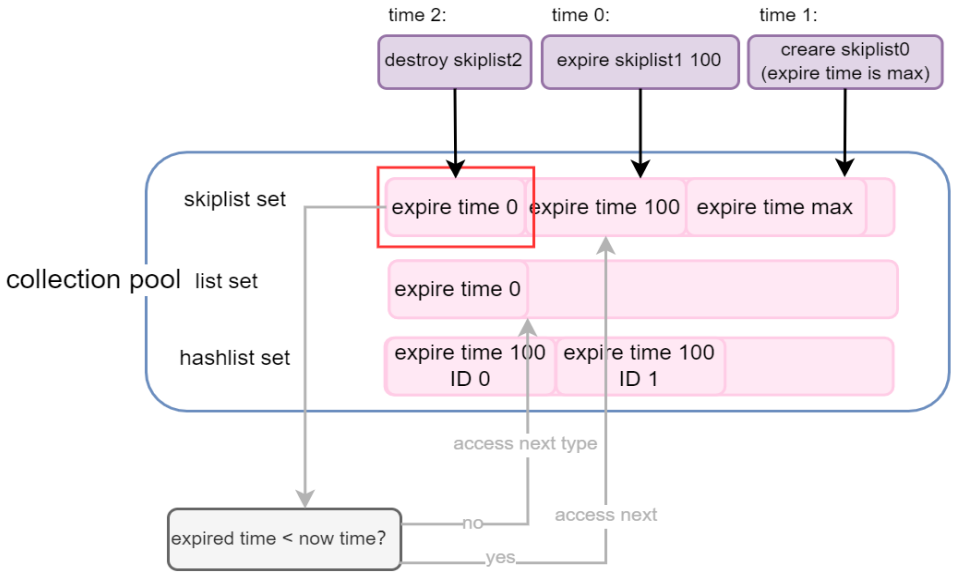

Figure 4

2. 前台用户线程负责更新 Collection Pool，后台 Cleaner 线程扫描 Collection Pool 找到过时 collection，然后将其从 Collection Pool 移入后台过时队列中。前后台线程都是加锁

更新/扫描 Collection Pool，为了避免加锁造成前台抖动延迟，Cleaner 每轮扫描 Collection Pool 最多可获取的过时 collection 数目的上限是用户设置的 Cleaner 最大线程数（max_thread_num）。因为我们假定用户的两种极端场景：一种是少量且元素较多的大 collection，另一种是大量且元素较少的小 collection。无论哪种场景，当有批量过时 collections 时，Cleaner 都是开启多线程模式处理，而且每个线程每次只销毁一个过时 collection，销毁过时 collection 之后再扫描 Collection Pool 获取新一批的过时 collection。参见 Figure5。

3. 值得注意的是，由于 KVDK 支持多版本查询功能，所以后台过时队列中的 collection 只有不存在任何历史版本中，才可以物理删除（释放空间）。考虑到用户设置 snapshot 是随着现实时间递增的，因此后台过时队列被设计为根据找到 outdated collection 的时间从小到大排序以减少 fetch outdated collection 的时间开销。为什么是找到 outdated collection 的时间，而不是 collection 的创建时间或者其他时间呢？思考一个具体的例子：t1 时刻创建了 skiplist collection； t2 时刻设置 skiplist collection 过期时间：在 t4 时刻过期， 同时其他线程在 skiplist collection 中添加了 100 个元素；t3 时刻用户设置 snapshot； t5 时刻用户基于 snapshot 访问 skiplist collection。如果后台线程在 t4 时刻发现 skiplist collection 已经过期，然后把它从 Hash Table 删除，然后再执行物理删除，那么用户在 t5 时刻是无法访问到不存在的 skiplist collection 因此为了避免上述情况，我们暂时将发现 outdated collection 的时间作为是否可以安全物理删除的判断标准。当此时间比最小的 snapshot 创建时间小时（表示该 outdated collection 不存在于任何 snapshot 中），才能将其从 Hash Table 上删除，然后再释放空间。当用户创建的 snapshot（历史版本）过多时，上述方案可能会导致大量空间无法及时释放。后续将继续优化这里的算法逻辑。

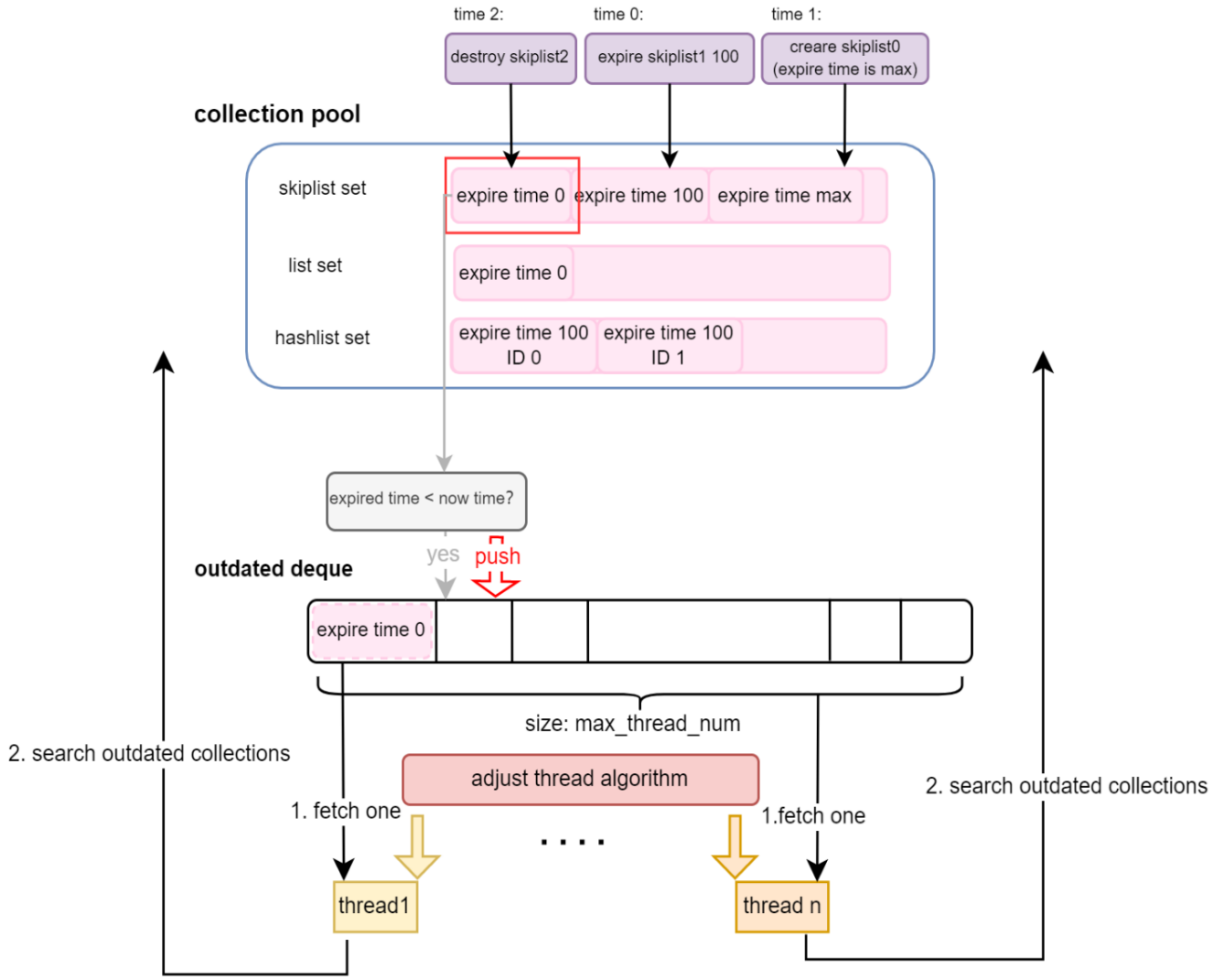

Figure 5

2.4 释放内存空间

无论是 String 类型数据还是 Collection 类型的数据元素，只有该内存空间释放了其内存空间，才表示该数据已经被真正的删除了。前面三个小节着重如何发现过时数据，以及何时从摘除该过时数据（不存在于任何用户设置的 snapshot 上，才可以从 Hash Table 摘除或从链表上摘除）。本小节将说明，何时释放该内存空间。

在 2.1 节中，我们解释了“为什么从版本链上摘除后，不能直接释放其空间”。这个原则对任何一个过时数据都是有效的。当一个 string 类型数据从 Hash Table 上摘除后，如果空间立即被释放并很快被重用，此时有个读线程正在访问该块内存，非常有可能会导致读到非法数

据，也就是常遇到的读写冲突问题。因此释放空间的原则为：

$$TimeStamp(purge\ outdated\ data) < Min\ \{token_0, \dots, token_n\};$$

$$TimeStamp(purge\ string\ outdated) = TimeStamp(Hash\ Table.Erase(string\ outdated\ data))$$

$$TimeStamp(purge\ sorted\ outdated\ data) = TimeStamp(Skiplist.Remove(sorted\ outdated\ data))$$

$$TimeStamp(purge\ outdated\ collection) = TimeStamp(Hash\ Table.Erase(outdated\ collection)$$

式子中的 Token 表示某个写线程拥有的本地的 token (snapshot)。

2.5 Cleaner 动态调整线程算法

Cleaner 最初只有一个后台主线程在扫描，采用轮询调度算法每次扫一个 slot block，即 1024 slot。主线程每扫完一个 slot block 后，会计算该 slot block 中过时数据个数的比例（outdated records ratio）。

$$outdated\ records\ ratio\ = \frac{need\ clean\ record\ num}{totol\ scan\ record\ num};$$

主线程扫描 Collection Pool 时，由 2.3 节中可知，扫描一轮 Collection Pool，最多找到 max cleaner thread num 个数的过时 collections，因此可得到 outdated collections ratio 为

$$outdated\ collections\ ratio\ = \frac{outdated\ collections\ num}{\max\ cleaner\ thread\ num}$$

outdated records ratio 和 outdated collections ratio 相加即可得总 outdated ratio：

$$outdated\ ratio\ =\ outdated\ records\ ratio + outdated\ collections\ ratio$$

如果该比例$outdated\ \ ratio < wakeup\ threshold$;， 表明需要清理的空间较少，Cleaner 的主线程 sleep 1 秒； 否则会根据下列公式计算出后台 Cleaner 需要多少线程清理空间：

$$\text{advice thread num} =\ \min(\max(outdated\ ratio * \text{max_thread_num}, 1)\ , \text{max_thread_num})$$

值得注意的是：计算总 outdated ratio 时，我们只统计了 Cleaner 扫描 Hash Table 过程中找到的过时数据和扫描 Collection Pool 过程中找到的过时 collection，并没有统计后台获取到的前台线程缓存队列中的过时数据。这是因为前台线程缓存队列中的数据不属于后台 Cleaner 扫描得到的，很难计算比例，而且当前台频繁写操作时，其缓存队列中的过时数据可以很快被清理掉。

3 Cleaner 性能分析

3.1 KVDK Benchmark

测试参数

- VALUES SIZE： 120B
- THREAD： 48

机器配置

- MOLDEL NAME： Intel(R) Xeon(R) Platinum 8260L CPU @ 2.40GHz
- CORE(S) PER SOCKET： 24
- CPU SOCKET： 2
- DRAM SOCKET： 64GB*6
- PMEM SOCKET： 126.422GB*6
- KERNEL VERSION： Linux 5.4.32

性能结果

- Baseline Commit Version： d2017795a07b0cd295ceae3a531af45934051b40 Author： JiayuWu
- Previous Commit Version： 58b0ae1eaa5cf94be9737c1d3f4eb61fa7679e50 Author： ZiyanShi
- Current Commit Version： cd9b756c8ebb8f607240016e25900c29aff80802 Author： Zhichen J

string	Baseline(Ops)	Previous(Ops)	Current (Ops)	Current vs Previous Rate(%)
point-read	42,158,642	42,323,678	42,423,428	0.24%
update	12,398,142	9,498,285	9,271,714	-2.39%
batch	17,632,888	16,760,857	16,751,892	-0.05%
insert	20,405,956	21,129,608	21,122,000	-0.04%
9R:1W	37,697,700/1,286,100	38,309,800/755,700	38,394,300/740,100	0.22%/-2.06%

Table 1: string benchmark

sorted	Baseline(Ops)	Previous(Ops)	Current (Ops)	Current vs Previous Rate(%)
point-read	30,363,892	30,316,392	30,433,678	0.39%
update	6,792,857	6,209,892	6,206,178	-0.06%
range	54,120,209	56,549,028	56,430,016	-0.21%
insert	4,430,642	4,458,785	4,525,071	1.49%
9R:1W	27,411,200/686,533	27,350,533/598,933	27,432,266/588,066	0.3%/-1.81%

Table 2: sorted benchmark

3.2 KVDK Cleaner 清理 records 性能

Table 2 展示了 Cleaner 清理 records 的性能。根据之前章节可知，Cleaner 通过扫描 Hash Table 找到过时数据，该过时数据主要是由过期或者删除的 string 类型和被删除的 skiplist 中的元素。Table 2 中的展示了单线程轮询扫描 Hash Table 的性能数据，“Find and purge outdated records“表示每秒找到并从逻辑上摘除（Hash Table 删除，skiplist 链表删除）的过时数据的个数。”Physically free space“表示每秒物理删除（释放内存空间）过时数据的个数。可以发现找到过时数据的性能（producer）和真正释放内存空间的性能（consumer）近乎相近。由于摘除 skiplist 元素需要更改前后指针，比 string 类型多了两次写，因而 ops 保持在 300K。Cleaner 单线程几百 K ops 清理速度可以满足”快速释放内存“的要求么？当然是可以的。前台单线程写 128B value 时，string 类型的写性能在 950K ops，skiplist 的写性能在 230K ops 左右。所以当用户将前后台最大线程数设置为相等时，后台 Cleaner 的性能可以达到 KVDK 的写性能。但大多数场景是更新操作较多，此时，Cleaner 最大线程数不需要与前台线程数一样。

DataType	Find and Purge outdated records (ops)	Physically free space (ops)
string	≈940K	≈940K
skiplist Elem	≈300K	≈300K

Table 3: During scanning Hash Table

3.3 KVDK Cleaner 清理 collections 性能

Table 4 展示了单线程删除 Collection 类型的性能。Cleaner 的每个线程每次也是从后台过时 collection 队列中获取一个 outdated collection，然后独立负责删除过时 collection。实际上等同于单线程删除 collection。如果删除的 collection 中含有较多元素，该线程会一直抢占 CPU 资源，对于单 core 用户场景，那将会是非常糟糕的。因此未来将会支持多线程删除 collection 来加速性能，同时也可以结合线程唤醒机制（类似于章节 2.4 中提到的 sleep 机制）来缓解后台线程抢占 CPU 资源。

DataType	Destroy collection(ops)
skiplist collection	610K
list collection	1M
hashlist collection	700K

Table 4: Destroy Collection Performance

4 Cleaner 未来优化点

1. 单线程销毁大 collection 的性能，以 Skiplist 为例，性能为 100M~200M/s。后台线程可能会卡死。之后可以考虑多线程销毁大 collection，来提升性能和快速释放空间。
2. 如 2.3 节提到的当用户设置了很多的 snapshot 时，需要一种机制能够清理 snapshots 之间不会再被访问的过时的数据。
3. 当用户单 core 单线程使用 KVDK 时，某段时期执行了大量写操作后活跃度下降，此期间内后台主线程正在清理一个大 collection，如果用户突然又开始执行大量写操作，由于 CPU 资源一直被后台线程占满，前台可能会出现卡顿。之后可能会根据客户场景考虑针对单 core 的优化。